Ukrainian Catholic University

Faculty of Applied Sciences

Business Analytics Program

Comparison of the Diffusion Models for Text-to-Image Generation

MMML Final Project Report

Authors:
Yuliia Vistak
Anastasiia Dynia
Viktoriia Stetsyshyn

07 April 2025

Abstract

Contents

1 Introduction

Imagine typing words and watching them turn into colourful and realistic images, every word transformed into art on your screen. We live in the era where words no longer only speak—they visually unfold, linking imagination with reality. This fascinating connection between text and image has attracted interest far and wide, including *authors' own.*

At the core of translating simple text into lifelike visuals are powerful **text-to-image models** and tools such as DALL-E, Midjourney, and Stable Diffusion, which have lately become extremely popular. These models first process text by converting it into high-dimensional embeddings, capturing its semantic meaning. Next, through advanced deep learning methods, they map these text embeddings to visual patterns, generating images from simple text prompts.

One of the most promising approaches among these methods is the use of **diffusion models**. Diffusion models, also known as diffusion probabilistic models, are a class of generative models that gradually add noise to data using a *forward process*. Once this noised (or "diffused") state is reached, the *reverse process* is learned—essentially reversing the diffusion—to recover the original data structure and generate new samples.

In this study the authors aim to describe different diffusion models, such as Denoising Diffusion Probabilistic Models, Denoising Diffusion Implicit Models, and Latent Diffusion Models, that can be used in transforming text into images.

Denoising Diffusion Probabilistic Models is one of the main and most common classes of generative models. The process of image generation consists of two stages. Firstly, the model processes a high-quality photo that has its own structure (e.g., some colors appear more frequently), forming a distribution with different outliers and patterns. The idea of this forward-process is to apply Gaussian noise on the image step by step, transforming the data distribution into a standard normal distribution, and train the model to predict the added noise on the photo. While it is hard to recognise the old version of the input image, the reverse process begins. All the noise is removed iteratively with the trained model, steadily restoring an image that is close to the original data.

Since image generation requires significant computational power, Denoising Diffusion Probabilistic Models have been modified into **Denoising Diffusion Implicit Models**. The forward stage appears to be the same in both models, but the key difference lies in the denoising process. Because DDPMs are based on a Markov chain, they require each step of the reverse process to remove the added noise sequentially. In contrast, DDIMs use a non-Markov process, which allows them to jump through certain steps, directly predicting the denoised image at a later time step.

Last but not least, the class of models, which would be described in this research is **Latent Diffusion Models**. While both previously described models work on pixel space, LDMs bring a new optimization approach by performing the diffusion process in the lower dimensional latent space instead of full-resolution pixel space. This approach is the basis of Stable Diffusion and significantly reduces computational complexity while ensuring high quality output. Primarily, this method uses the autoencoder, which transforms pixel space into latent space. Then the whole diffusion process is applied to the new lower dimensional space of the image and the decoder reconstructs the final image in the pixel space.

The link to our solution is: Colab Notebook

2 Method Discussion

2.1 Denoising Diffusion Probabilistic Models

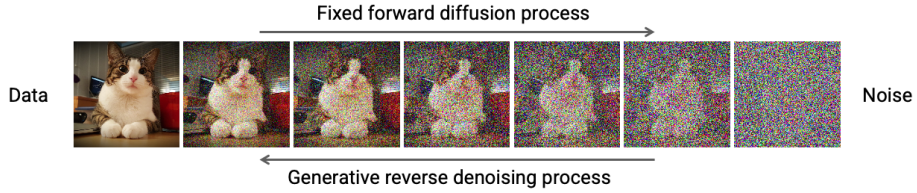

Figure 1: Diffusion process. Source

A **Denoising Diffusion Probabilistic Model** is a parameterized Markov chain trained using variational inference that maps data to a simple distribution and generates data-like samples within a finite number of steps. The model learns the transition kernels in a reversed direction of diffusion (forward process), which itself is a Markov chain that adds noise to the data, destroying the original sample (image) [3]. The diffusion processes are shown in Figure 1.

2.1.1 Forward (Diffusion) Process

As stated above, forward process is a Markov chain that gradually adds Gaussian noise to the data over a sequence of T steps, effectively destroying the original structure of the data.

$$x_0 \to x_1 \to \cdots \to x_T$$

Given a data distribution $x_0 \sim q(x_0)$, the forward process generates a sequence of increasingly noisy data points $(x_1, x_2, \ldots, x_T)$ using the following formula:

$$q(x_{1:T} \mid x_0) := \prod_{t=1}^{T} q(x_t \mid x_{t-1}), \tag{1}$$

where each transition is defined as a Gaussian:

$$q(x_t \mid x_{t-1}) := \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t \mathbf{I}). \tag{2}$$

Here,

- β_t is a variance schedule that controls how much noise is added at each step.
- $\mathcal{N}$ denotes a multivariate normal distribution.

Using the property that any arbitrary step of the forward process can be sampled directly in closed form, define:

$$\alpha_t := 1 - \beta_t, \quad \bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s.$$

According to these definitions, the marginal distribution of x_t conditioned directly on x_0 can be expressed as:

$$q(x_t \mid x_0) := \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1 - \bar{\alpha}_t)\mathbf{I}). \tag{3}$$

This closed-form allows sampling noisy data at any step t without evaluating the entire chain.

2.1.2 Reverse Process

Reverse process is trained to remove the noise step by step and recover a data sample from pure noise. This process is modeled as a learned Markov chain with Gaussian transitions.

$$x_T \to x_{T-1} \to \cdots \to x_0$$

The joint distribution $p_\theta(x_{0:T})$ defines the reverse process, starting from a known prior $p(x_T) = \mathcal{N}(x_T; 0, \mathbf{I})$:

$$p_\theta(x_{0:T}) := p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \tag{4}$$

$$p_\theta(x_{t-1} \mid x_t) := \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)). \tag{5}$$

Here:

- $\mu_\theta(x_t, t)$ and $\Sigma_\theta(x_t, t)$ are estimates of the mean and variance of the denoising distribution.

Formally, diffusion models are defined as:

$$p_\theta(x_0) := \int p_\theta(x_{0:T}), dx_{1:T}, \tag{6}$$

To calculate $p_\theta(x_0)$ it is necessary to marginalize over all the possible trajectories one could arrive to x_0 when starting from a noise sample. But unfortunately, it is contractable.

The solution is to maximize a lower bound, meaning to train the model by optimizing the **Evidence Variational Lower BOund (ELBO)**.
Consider the following.

- Latent variables: $x_1, x_2, \ldots x_T$
- Observed variable: x_0

Variational lower bound of log-likelihood:

$$\log p_\theta(x) \geq \mathbf{E}_q \left[\log p_\theta(x_0 \mid x_{1:T})\right] - D_{\mathrm{KL}}(q(x_{1:T} \mid x_9) \,\|\, p_\theta(x_{1:T})) =$$

$$= \mathbf{E}_q \left[\log p_\theta(x_0 \mid x_{1:T})\right] - \mathbf{E}_q \left[\log \frac{q(x_{1:T} \mid x_0)}{p_\theta(x_{1:T})}\right] =$$

$$= \mathbf{E}_q \left[\log p_\theta(x_0 \mid x_{1:T}) + \log \frac{p_\theta(x_{1:T})}{q(x_{1:T} \mid x_0)}\right] =$$

$$= \mathbf{E}_q \left[\log p_\theta(x_T) + \sum_{t \geq 1} \log \frac{p_\theta(x_{t-1} \mid x_t)}{q(x_t \mid x_{t-1})} \right] =$$

$$= \mathbf{E}_q \Big[- D_{\mathrm{KL}}(q(x_T \mid x_0) \,\|\, p(x_T)) \\ - \sum_{t>1} D_{\mathrm{KL}}(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)) + \log p_\theta(x_0 \mid x_1) \Big] \tag{7}$$

In practice, DDPM simplifies this objective by predicting the noise ϵ added at each step. The training loss becomes [4]:

$$\mathbf{E}_{t \sim \mathcal{U}(1,T),, x_0 \sim q(x_0),, \epsilon \sim \mathcal{N}(0,\mathbf{I})} \left[\lambda(t) \left\| \epsilon - \epsilon\theta(x_t, t) \right\|^2 \right], \tag{8}$$

where:

- $\epsilon_\theta(x_t, t)$ is the model's prediction of the noise.
- $\lambda(t)$ is a weighting function (often set as 1).
- x_t is sampled using the closed-form forward process: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$.

2.2 Denoising Diffusion Implicit Models (DDIM)

The main problem of DDPM is that there is not a single path that can go from x_t to x_0. In addition, the process of generating images with DDPM is quite long, so people came up with a faster approach that speeds up the process without sacrificing quality. DDIM brought improvements by making the inverse process deterministic, clearly and predictably removing noise.

2.2.1 Forward Process

The main idea of this process is to generate a noisy version of x_t of the original image x_0 at an independent moment of time t in one step, without iterative addition of noise. This formulation allows for a deterministic inverse process, where x_{t-1} can be be calculated from x_{t-1} and estimate of x_0.

$$q_\sigma(x_{1:T} \mid x_0) = q_\sigma(x_T \mid x_0) \cdot \prod_{t=2}^{T} q_\sigma(x_{t-1} \mid x_t, x_0) \tag{9}$$

- $q_\sigma(x_T \mid x_0)$ - indicates that it is possible to add noise to the clean image x_0 in a single step using the following formula:

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t}\, \epsilon, \quad \text{where} \quad \epsilon \sim \mathcal{N}(0, I) \tag{10}$$

- $\sqrt{\alpha_t}\, x_0$ – the preserved part of the original image.
- $\sqrt{1 - \alpha_t}\, \epsilon$ – the noise part that is added at time step t.

- $\prod_{t=2}^{T} q_\sigma(x_{t-1} \mid x_t, x_0)$ - represents the full forward diffusion path as a sequence of conditional Gaussian distributions. Each individual distribution $q_\sigma(x_{t-1} \mid x_t, x_0)$ determines the probability of getting a noisy image at step $t-1$ given the current noisy image x_t and the initial clean image x_0. As a result, this product accumulates the effect of each transition, forming a complete sequence of noise addition at all stages.

In contrast to DDPM, which is based on the Markov chain approach and considers each transition to depend only on the previous time step, DDIM relies on non-Markovian approach. [5]It is achieved by using information about x_0 and x_t at each time step, which allows for direct transitions without stochastic sampling.

2.2.2 Reverse Process

In the reverse process, the goal is to recover the image x_0 starting from a noisy image x_t. However, since during generation there is no x_0, the estimation is used.

$$f_\theta^{(t)}(x_t) = \frac{x_t - \sqrt{1-\alpha_t} \cdot \epsilon_\theta(x_t)}{\sqrt{\alpha_t}} \tag{11}$$

It comes from the following formulas:

$$\begin{aligned} x_t &= \sqrt{\alpha_t} x_0 + \sqrt{1-\alpha_t} \cdot \epsilon \\ x_0 &= \frac{x_t - \sqrt{1-\alpha_t} \cdot \epsilon}{\sqrt{\alpha_t}} \\ x_0 &\approx f_\theta^{(t)}(x_t) \end{aligned} \tag{12}$$

Using the estimated x_0, the reverse distribution is computed as:

$$p_\theta^{(t)}(x_{t-1} \mid x_t) = q_\sigma(x_{t-1} \mid x_t, x_0 = f_\theta^{(t)}(x_t)) \tag{13}$$

The case is slightly different at timestep $t = 1$, when the final denoised image x_0 is reconstructed. At this point $f_\theta^{(1)}(x_1)$ is the final estimate of the clean image,which is considered to be the result of generation.

To summarize, the reverse process is defined as:

$$p_\theta^{(t)}(x_{t-1} \mid x_t) = \begin{cases} \mathcal{N}(f_\theta^{(1)}(x_1), \sigma_1^2 \mathbf{I}) & \text{if } t = 1 \\ q_\sigma(x_{t-1} \mid x_t, f_\theta^{(t)}(x_t)) & \text{otherwise} \end{cases} \tag{14}$$

2.2.3 Loss Function

During the image generation, at each step t, the distribution over x_{t-1} conditioned on x_t and x_0 as a Gaussian is modeled. However, during this process, there is not access to the original image x_0. Therefore, the neural network to predict the noise ϵ_θ and reconstruct an estimate of x_0 is used. To evaluate the loss between the true and predicted distributions KL-divergence is used:

$$\sum_t \log \frac{p_\theta(x_{t-1} \mid x_t)}{q_\sigma(x_{t-1} \mid x_t, x_0)}$$

Both distributions have the same variance σ_t^2, but different means. The true mean corresponds to using real noise ϵ, while the predicted mean uses the neural network's estimate ϵ_θ.

The mean predicted by the neural network is:

$$\mu_\theta = \frac{x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \cdot \epsilon_\theta}{\sqrt{\alpha_t}}$$

The true mean used in the forward process is:

$$\mu = \frac{x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \cdot \epsilon}{\sqrt{\alpha_t}}$$

Since both are Gaussians with the same variance but different means, a specific form of KL-divergence can be applied for this case.

$$D_{\mathrm{KL}}(\mathcal{N}(\mu_q, \Sigma_q) \| \mathcal{N}(\mu_\theta, \Sigma_q)) = \frac{1}{2}(\mu_q - \mu_\theta)^T \Sigma_q^{-1} (\mu_q - \mu_\theta)$$

where

$$\Sigma_q = \sigma_q^2 \mathbf{I}, \quad \Sigma_q^{-1} = \frac{1}{\sigma_q^2}\mathbf{I}$$

$$(\mu_q - \mu_\theta)^T(\mu_q - \mu_\theta) = \|\mu_q - \mu_\theta\|^2 \quad \Rightarrow \quad D_{\mathrm{KL}} = \frac{1}{2\sigma_q^2}\|\mu_q - \mu_\theta\|^2$$

$$\mu_q - \mu_\theta = \left(\frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}\sqrt{\alpha_t}}\right)(\epsilon_\theta - \epsilon)$$

$$D_{\mathrm{KL}} = \frac{1}{2\sigma_q^2} \cdot \left(\frac{(1-\alpha_t)^2}{(1-\bar{\alpha}_t)\alpha_t}\right) \cdot \|\epsilon_\theta - \epsilon\|^2 = \frac{1-\alpha_t}{2\sigma_q^2 \alpha_t} \cdot \|\epsilon_\theta - \epsilon\|^2 \tag{15}$$

2.2.4 Impact of Variance in DDIM

The variance σ_t plays a key role in defining the reverse sampling process.

When $\sigma_t = 0$, the process becomes fully deterministic. The same image x_0 will always result in the same sampled image x_t because the same noise is added at every timestep. The only source of randomness is the initial noise added to x_0.

In contrast, DDPM introduces new random noise at every step, which naturally provides stochasticity and enables diverse generations.

Therefore, to generate different outputs with DDIMs, the initial noise ϵ has to vary, which can be controlled by changing the value of σ_t. The higher the value of σ_t, the more stochasticity is introduced into the process.

There is a specific non-zero σ_t^2, which is also used in DDPM for the true posterior distribution of the forward process:

$$\sigma_t^2 \to \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} \tag{16}$$

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\mu_q, \sigma_t^2 \mathbf{I})$$

Thus, DDIM allows flexible control over the level of stochasticity: it can be fully deterministic when $\sigma_t = 0$, or correspond to the stochastic behaviour of the DDPM.

2.3 Latent Diffusion Models

2.3.1 Overview of LDM

The **high-dimensional nature of image data** presents significant challenges for generative modeling. Pixel-based image representations inherently contain numerous high-frequency details, many of which are barely perceptible to human observers.

Diffusion Probabilistic Models have recently emerged as state-of-the-art techniques in density estimation and sample quality. However, when directly operating in pixel space, these models effectively act as lossy compressors, trading image quality against compression efficiency. Evaluating and optimizing diffusion models in pixel space also include *low inference speeds* and *high training costs*, especially with high-resolution data.

To overcome these limitations, the authors propose to work with **Latent Diffusion Models** (LDMs) [1]], which operate within a compressed latent space of significantly lower dimensionality. By sampling within this compressed latent representation rather than directly in pixel space, we achieve substantial improvements in computational efficiency, significantly lowering training costs and improving inference speed without compromising synthesis quality.

2.3.2 Latent Space Representation

Before delving into latent spaces and autoencoders, it's helpful to recall how images are stored in computers. Grayscale images are represented as a grid of pixels, with each pixel holding a value from 0 to 255 that indicates its brightness—0 for black and 255 for white. Colored images follow a similar format, but they consist of three channels (red, green, and blue), each ranging from 0 to 255.

In contrast to this high-dimensional pixel representation, autoencoders learn to compress images into a **lower-dimensional latent space**. This latent space retains the essential, perceptually significant features of the original image while removing redundant information.

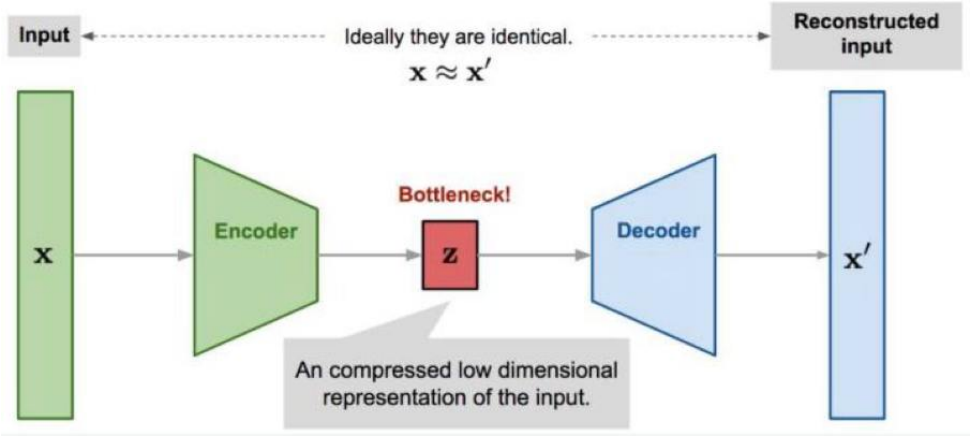

Figure 2: Autoencoder architecture.
Source

The encoder $\mathcal{E}$ performs this compression by mapping the image x to a latent representation z, expressed mathematically as:

$$z = \mathcal{E}(x),$$

where z has a shape $(h \times w \times c)$ with h and w being significantly smaller than H and W, and c representing the number of feature channels in the latent space. Typically, the encoder down-samples the spatial dimensions by a factor f, so that

$$f = \frac{H}{h} = \frac{W}{w}.$$

In practice, f is often chosen as a power of 2, such as $f = 2, 4, 8$, etc., which allows for efficient computation.

The decoder $\mathcal{D}$ then reconstructs the image from the latent representation:

$$\tilde{x} = \mathcal{D}(z),$$

with the goal that $\tilde{x}$ is as close as possible to the original x in terms of perceptual quality.

2.3.3 Autoencoder Training and Regularization

To obtain compact latent representations suitable for diffusion modeling, the authors first train a convolutional autoencoder consisting of an encoder $\mathcal{E}$ and decoder $\mathcal{D}$ pair. Unlike adversarial or quantized setups, the design follows a simpler reconstruction-based training objective using a standard mean squared error (MSE) loss:

$$L_{\text{rec}} = E_{x\sim\mathcal{D}}\left[\|x - \mathcal{D}(\mathcal{E}(x))\|_2^2\right].$$

This formulation encourages the model to learn lossy but consistent latent encodings of MNIST digits without the need for additional regularization mechanisms such as KL divergence or vector quantization.

The encoder $\mathcal{E}$ compresses an input image $x \in R^{1\times 28\times 28}$ into a 16-dimensional vector $z \in R^{16}$, which is then decoded back to image space by $\mathcal{D}$. This bottleneck structure naturally enforces regularity in the latent space, while the absence of adversarial or probabilistic terms allows the autoencoder to prioritize accurate reconstructions over generative flexibility.

As such, no explicit latent-space regularization term L_{reg} is used. The training objective reduces to:

$$L_{\text{autoencoder}} = \min_{\mathcal{E},\mathcal{D}} L_{\text{rec}}(x, \mathcal{D}(\mathcal{E}(x))).$$

2.3.4 Training the Diffusion Models in Latent Space

Once the autoencoder is trained, its encoder $\mathcal{E}$ is used to project all dataset images into a fixed latent space $\mathcal{Z} \subset R^{16}$. A conditional Denoising Diffusion Probabilistic Model (DDPM) is then trained directly on these low-dimensional latent vectors.

Latent Space Properties. Since the latent space is not explicitly regularized using a KL or VQ objective, the learned representations do not follow a pre-specified distribution. Nevertheless, empirical analysis suggests that the latents are bounded and approximately

centered. To improve numerical stability during training, the latent vectors are normalized within each batch. Let

$$\hat{\mu} = \frac{1}{B}\sum_{i=1}^{B} z_i, \quad \text{and} \quad \hat{\sigma}^2 = \frac{1}{B}\sum_{i=1}^{B}(z_i - \hat{\mu})^2, \tag{17}$$

where $z_i = \mathcal{E}(x_i) \in R^{16}$ is the encoded latent for sample x_i in a batch of size B. The normalized latent z_{scaled} is then computed as:

$$z_{\text{scaled}} = \frac{z - \hat{\mu}}{\hat{\sigma}}. \tag{18}$$

Diffusion Objective. The diffusion model is trained to recover clean latents from noisy ones using a time- and text-conditioned denoiser. Each training step proceeds as follows:

1. Encode the image x into a latent vector $z_0 = \mathcal{E}(x)$.
2. Sample a timestep $t \sim \mathcal{U}\{1, \ldots, T\}$, with $T = 1000$.
3. Add Gaussian noise to obtain $z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, I)$.
4. Condition on the associated text prompt and train the denoising model to predict ε.

This yields the following training objective:

$$L_{\text{LDM}} = E_{z_0, t, \varepsilon \sim \mathcal{N}(0,I)} \left[\|\varepsilon - \varepsilon_\theta(z_t, t, e_y)\|_2^2 \right], \tag{19}$$

where e_y is the embedding of the text prompt and ε_θ is the time- and text-conditioned noise estimator.

Sampling and Reconstruction. During inference, latent vectors $z_T \sim \mathcal{N}(0, I)$ are sampled and reverse diffusion steps are applied iteratively from $t = T$ to $t = 0$ using the trained denoiser. The final clean latent z_0 is then decoded as:

$$\tilde{x} = \mathcal{D}(z_0),$$

to generate the synthetic image conditioned on the input text. As no vector quantization is employed, the reconstruction remains continuous and fully dependent on the learned autoencoder decoder.

3 Application Domains

Earlier, the authors discussed the fundamentals of image generation using diffusion models and latent diffusion models. In general, there are two broad categories of image generation applications using diffusion models: unconditional and conditional. Both approaches have distinct benefits and can be applied in a variety of settings.

Unconditional Image Generation

Unconditional diffusion models generate images without any external guidance. They have demonstrated impressive capabilities in synthesizing high-quality images from random noise. Some applications include:

- **Creative Art Synthesis:** These models can produce novel and visually striking artworks, often used for artistic exploration and design.
- **Data Augmentation:** Unconditional models can generate diverse images that enrich training datasets, particularly in scenarios where labeled data is scarce.

Conditional Image Generation

Conditional diffusion models extend the capabilities of unconditional generation by incorporating additional information to guide the synthesis process. Some notable applications are:

- **Class-Conditioned Generation:** By conditioning on class labels, the model can generate images that belong to a specific category, useful for tasks such as dataset balancing and targeted content synthesis.
- **Text-to-Image Synthesis:** Perhaps the most intuitive form of conditioning, where text prompts guide the generation process, producing images that align with the semantics of the given description.

Text-to-image generation

The goal of text-to-image generation is to synthesize an image that semantically corresponds to a given natural language prompt. In the case of testing on the MNIST dataset, simple digit prompts, such as *"a photo of digit 3"*, are considered.

The image generation process starts by encoding the text prompt into vector representations, which are then integrated into the diffusion model using cross-attention. At each step, the model uses these representations to guide denoising and gradually generate an image that matches the prompt.

1. Text prompt representation

Firstly, the input prompt y is first split into a sequence of n tokens:

$$y \rightarrow [t_1, t_2, \ldots, t_n] = [\text{"a"}, \text{"photo"}, \text{"of"}, \text{"digit"}, \text{"three"}]$$

Then, each token t_i is mapped to a learnable embedding vector $\mathbf{e}_i \in R^d$ and a position vector $\mathbf{p}_i$ is added to each $\mathbf{e}_i$ to encode word order:

$$\mathbf{x}_i = \mathbf{e}_i + \mathbf{p}_i$$

Finally, the sequence $[\mathbf{x}_1, \ldots, \mathbf{x}_n]$ is passed through a Transformer encoder to produce contextualized token representations:

$$\mathbf{z}_1, \ldots, \mathbf{z}_n \in R^d$$

Each vector $\mathbf{z}_i$ now captures the meaning of token t_i in the context of the entire prompt.

2. Cross-attention integration

Now, to guide the image generation process, the model uses **cross-attention** to relate image features to the textual prompt. At each denoising step, an intermediate image feature vector q (query) interacts with the sequence of text embeddings $\tau(y) = [\mathbf{z}_1, \ldots, \mathbf{z}_n]$.

This is achieved by projecting:

- the query q into a query vector $Q = W_Q \cdot q$,
- each token embedding $\mathbf{z}_i$ into a key $K_i = W_K \cdot \mathbf{z}_i$,
- and into a value $V_i = W_V \cdot \mathbf{z}_i$.

The similarity between Q and each K_i is computed using scaled dot-product attention:

$$\alpha_i = \text{softmax}\left(\frac{Q \cdot K_i^\top}{\sqrt{d_k}}\right)$$

Each α_i indicates how relevant the i-th word in the prompt is to the current image feature q.

The model then computes a **context vector** as a weighted sum of the value vectors:

$$\text{Context}(q) = \sum_{i=1}^{n} \alpha_i \cdot V_i$$

This context vector is passed into the U-Net decoder and guides the image processing. For instance, if the word *"three"* is most semantically relevant, the corresponding V_i will dominate and influence the visual content being generated.

3. Conditional denoising objective

At training time, the model is trained to predict the noise ε added at each timestep t, given the noised image z_t and the text embedding $\tau(y)$. The loss function is:

$$\mathcal{L}_{\text{CDM}} = E_{x,y,\varepsilon,t}\left[\|\varepsilon - \varepsilon_\theta(z_t, t, \tau(y))\|_2^2\right]$$

Where:

- z_t is the noisy latent image at timestep t
- ε_θ is the model prediction of the noise
- $\tau(y)$ is the text conditioning signal

4. Generation Process

Once trained, the generation starts from random noise z_T and proceeds via reverse diffusion steps. On each step:

1. The model uses the current latent z_t, timestep t, and the text embedding $\tau(y)$
2. Through cross-attention, it focuses on relevant textual tokens
3. It predicts the noise and removes it from z_t
4. This continues until z_0, the final image, is reached

Example: For the prompt *"a photo of digit 3"*, the model learns to generate an MNIST-style image resembling the digit 3, as its attention focuses on the semantically rich word *"three"* during decoding.

4 Metrics

4.1 Frechet Inception Distance (FID)

Frechet Inception Distance (FID) is a widely used evaluation metric for assessing the quality of the generated images. It meausures hoe well the images are generated in the synthetic set and how realistic they seem to a human.

Let $I_S = I_{S_1}, I_{S_2}, \dots$ be the set of synthetic images and $I_R = I_{R_1}, I_{R_2}, \dots$ the set of real images.

The process of computing FID involves the following steps:

- Firstly, both the synthetic and real images are passed through a pre-trained InceptionV3 network, which is used to extract high-level feature representations from the images.
- The extracted feature representations are formnd into vectors of dimension 2048.
- Let M_S be the mean vector of the features of the generated images, and M_R be the mean vector of the features of the real images.
- Σ_S and Σ_R are the covariance matrices of the features of the generated and real images, respectively.

The FID score is then calculated with the following formula:

$$\text{FID}(M_S, M_R, \Sigma_S, \Sigma_R) = |M_S - M_R|_2^2 + \text{Tr}\left(\Sigma_S + \Sigma_R - 2\left(\Sigma_S \Sigma_R\right)^{\frac{1}{2}}\right) \tag{20}$$

where:

- $|M_S - M_R|_2^2$ measures the squared distance between the mean vectors.
- $\text{Tr}(\Sigma_S + \Sigma_R)$ measures the overall variance of each distribution.
- $-2\ \text{Tr}\left(\left(\Sigma_S \Sigma_R\right)^{\frac{1}{2}}\right)$ measures how similar the shapes of the two distributions are.

A lower FID score indicates that the synthetic images are more similar to the real images, meaning higher image quality and realism.

4.2 Inception Score (IS)

Inception Score (IS) is a metric designed to evaluate the efficacy of image generation models by assessing two critical properties: **diversity** and **fidelity**.

Diversity - the generated images should be varied across different classes.

Fidelity - each generated image should be semantically clear and recognizable to a human observer, and clearly belong to one of the classes.

Let Ω_x denote the space of images and Ω_y the space of the classes.

Given:

- $x \sim p_{gen}$, where p_{gen} is the distribution of generated images.
- $\mathcal{M}(\Omega_y)$ represents the space of all possible probability distributions over labels.

One of the steps involves passing an image x through a pre-trained classifier to obtain a probability vector indicating the likelihood of the image belonging to each class.

This process can be described by a classifier model:

$$\rho : \Omega_x \to \mathcal{M}(\Omega_y) \tag{21}$$

where $\rho(x)$ outputs the probability distribution over classes for the image x.

The Inception Score is defined as:

$$IS = \exp\left(E_{x\sim p_{gen}}\left[D_{KL}(p(y\,|\,x)\|p(y))\right]\right) \tag{22}$$

where:

- $D_{KL}(p(y\,|\,x)\|p(y))$ is the Kullback-Leibler divergence between the conditional label distribution $p(y\,|\,x)$ and the marginal distribution $p(y)$.
- $p(y) = E_{x\sim p_{gen}}[p(y\,|\,x)]$ is the marginal distribution over all generated images.
- $p(y\,|\,x)$ is the conditional probability distribution over classes given a generated image x; it indicates the classifier's predicted likelihood for each class.

4.3 Precision and Recall for Generative Models

Precision and Recall are important metrics for evaluating the performance of generative models. They measure how realistic the generated images are and how well they cover the diversity of real images.

- **Precision**: Measures how realistic the generated images are by checking whether they lie inside the manifold of real images.
- **Recall**: Measures how diverse the generated images are by checking whether the real images are covered by the manifold of generated images.

k-NN Precision/Recall Method

k-Nearest Neighbors (k-NN) based approach is used to estimate these metrics:

- Let X_r be the set of real images and X_g be the set of generated images.
- Extract features (embeddings) for all images by passing them through a feature extractor Φ (e.g., a convolutional neural network).
- Let $\Phi_r = \{\varphi_r^i\}_{i=1}^N$ and $\Phi_g = \{\varphi_g^i\}_{i=1}^M$ be the sets of feature vectors for real and generated images respectively.
- Assume $N = M$, meaning there is an equal number of real and generated samples.

Constructing the Manifold

- For each feature vector φ' in a set Φ, find the distance to its k-th nearest neighbor $NN_k(\varphi', \Phi)$.
- Define a local radius $r(\varphi')$ as:

$$r(\varphi') = \|\varphi' - NN_k(\varphi', \Phi)\|_2 \tag{23}$$

- Each feature vector thus defines a hypersphere, and the union of all hyperspheres forms the manifold.

Determining if a Point Belongs to a Manifold

Define a binary function $f(\varphi, \Phi)$:

$$f(\varphi, \Phi) = \begin{cases} 1, & \text{if } \exists \varphi' \in \Phi \text{ such that } \|\varphi - \varphi'\|_2 \leq r(\varphi') \\ 0, & \text{otherwise} \end{cases} \tag{24}$$

Precision and Recall Formulas

Precision is the fraction of generated samples that fall inside the manifold of real samples:

$$\text{Precision}(\Phi_r, \Phi_g) = \frac{1}{|\Phi_g|} \sum_{\varphi_g \in \Phi_g} f(\varphi_g, \Phi_r) \tag{25}$$

Recall is the fraction of real samples that fall inside the manifold of generated samples:

$$\text{Recall}(\Phi_r, \Phi_g) = \frac{1}{|\Phi_r|} \sum_{\varphi_r \in \Phi_r} f(\varphi_r, \Phi_g) \tag{26}$$

4. CLIP Score

CLIP Score is a metric that measures how well a generated image aligns with a given textual description. It leverages the CLIP model (Contrastive Language–Image Pre-training), which embeds both images and text into a shared latent space and computes their similarity using cosine distance.

The score is calculated as follows:

- Each generated image is embedded using the image encoder of the CLIP model.
- Each prompt is embedded using the text encoder.
- The cosine similarity between each image-text pair is computed.

A higher CLIP Score indicates that the generated image is more semantically aligned with the input text. This metric is particularly useful for evaluating models that are conditioned on prompts, as it reflects how faithfully the model interprets the textual input.

Formally, given a set of image embeddings $\{v_i\}$ and corresponding text embeddings $\{t_i\}$, the CLIP Score is computed as:

$$\text{CLIP Score} = \frac{1}{N} \sum_{i=1}^{N} \cos(v_i, t_i)$$

where $\cos(v_i, t_i)$ denotes the cosine similarity between image embedding v_i and text embedding t_i.

In this report, CLIP Score is used to assess the semantic coherence between the generated MNIST digits and the corresponding prompts. It complements other metrics such as FID and IS by focusing on prompt fidelity.

5 Experiments and Results

As part of this project, the authors implemented the **Denoising Diffusion Probabilistic Model (DDPM)**, **Denoising Diffusion Implicit Model (DDIM)** and **Latent Diffusion Model (LDM)** for the task of generating handwritten digits with a text description. The main goal of the experiment was to investigate how different types of textual prompts affect the quality of the generated images.

In particular, three variants of prompts,describing the same digits in different ways, were tested:

- **numeric** *'0', '1', ..., '9'*
- **digit**: *'digit 0', ..., 'digit 9'*
- **long**: *'a handwritten digit zero', ..., 'a handwritten digit nine'*

The models were trained separately for each type of prompt. After training, the results were evaluated both visually and quantitatively using commonly used generation quality metrics: **Fréchet Inception Distance (FID)**, **Inception Score (IS)**, **Precision, Recall, Density, Coverage (PRDC)** and **CLIP Score**, which were previously described.

5.1 DDPM & DDIM

Data and Prompt Setup

The model is trained on the MNIST dataset, which contains grayscale images of handwritten digits (28×28 pixels) with class labels from 0 to 9. For conditional generation, each digit is represented using one of the following types of text prompts:

- **numeric**: `"0", "1", ..., "9"`
- **digit**: `"digit 0", ..., "digit 9"`
- **long**: `"a handwritten digit zero", ..., "a handwritten digit nine"`

These text prompts are embedded using a pretrained transformer encoder (e.g., BERT), which produces a 256-dimensional vector representation used for conditional generation.

Model Overview

The DDPM/DDIM architecture consists of three main stages:

1. **Text Prompt → Text Encoder:** The input text is encoded into a vector embedding.
2. **Text Embedding + Noise Vector → Diffusion Denoising Model:** The embedding is passed into the diffusion model (DDPM or DDIM) together with a noisy image. The model gradually reconstructs the image using a U-Net conditioned on both timestep and text.
3. **Output → Generated Image:** The final output is a synthetic image generated in accordance with the input text prompt.

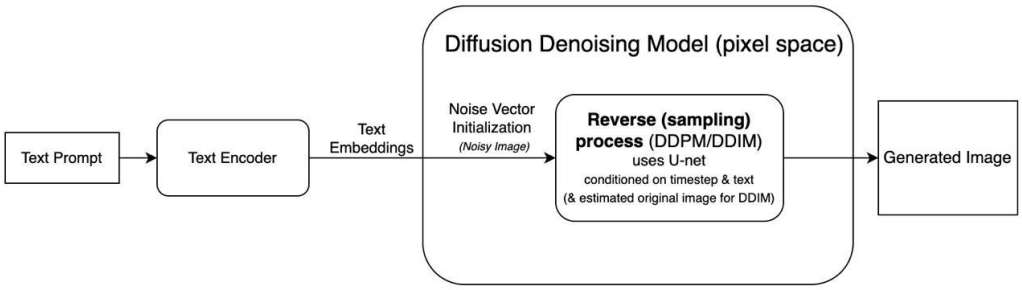

Figure 3: Architecture of the DDPM/DDIM model for text-to-image generation

Latent Diffusion Process

The generation process starts with noise sampled as $x_T \sim \mathcal{N}(0, I)$, and the reverse process denoises it over T steps using a U-Net conditioned on the text embedding c_y. In DDIM, the process is deterministic: each step estimates the original image $\hat{x}_0$, which is used to compute the next state in the trajectory from x_T to x_0.

Conditioning Mechanism

The model is conditioned on the embedded prompt c_y, which is concatenated with other inputs (such as timestep or latent noise) and fed into the U-Net for denoising.

Summary

To summarize, DDPM/DDIM is a text-conditioned generative diffusion model. It takes a textual description, encodes it, and uses that embedding to guide the denoising process that transforms noise into a meaningful image aligned with the text.

5.1.1 DDPM with long prompt

Let's look at the results of the trained DDPM model with prompts: *"a handwritten digit one"*.

This graph shows the change in the loss function over the training for both training and test samples.

There is a decrease in the loss during the first 16 epochs, which indicates that the model is learning effectively. Training was stopped at epoch 18 using 'early stopping' to avoid overfitting.

The visualisation of the generated long-prompt images demonstrates that the model is able to distinguish and generate handwritten digits according to the textual description. The digits are generally of a recognisable shape, although the quality varies: some characters appear clear (e.g., '0', '1', '4', '5', '7', '8'), while others may be less distinct or slightly distorted.

Let us now consider the results of the evaluation metrics:

- **FID Score:** 171.04
- **Inception Score:** 1.4758 ± 0.0410

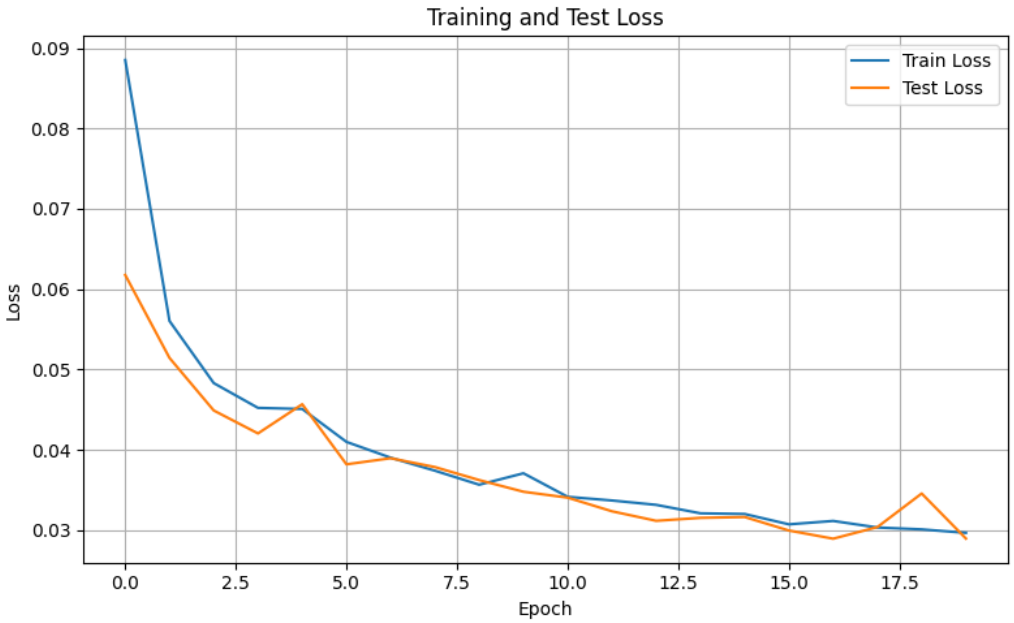

Figure 4: Loss during training DDPM with long prompt

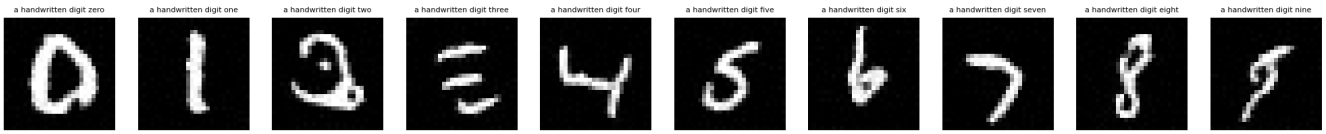

Figure 5: Photos sampled by DDPM with long prompt

- **PRDC:**
  - Precision: 0.0330
  - Recall: 0.0570
  - Density: 0.0192
  - Coverage: 0.0330
- **CLIP Score:** 23.99

The metrics show that **DDPM with long prompts** demonstrates the poor quality of generations: high **FID** (171.04), low **Inception Score** (1.4758 ± 0.0410), and very weak **PRDC** values, indicating fuzziness, low accuracy, and poor coverage of the generated images. Despite the length and informative nature of the prompts, the model was unable to effectively use the textual description. The **CLIP Score** (23.99) was relatively decent but could not compensate for the overall weak generation performance.

5.1.2 DDPM with digit prompt

Let's look at the results of the trained DDPM model with prompts like: *"digit one"*.

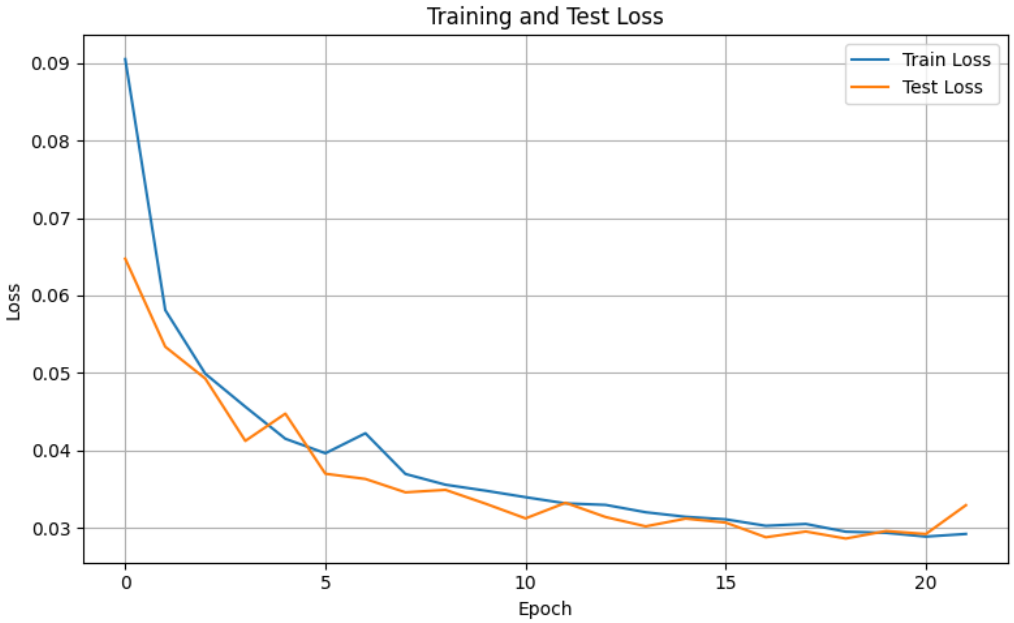

Figure 6: Loss during training DDPM with digit prompt

There is a clear decreasing trend in both training and test losses, which indicates stable and efficient model training. The difference between the curves is minimal, which indicates the absence of significant overfitting.

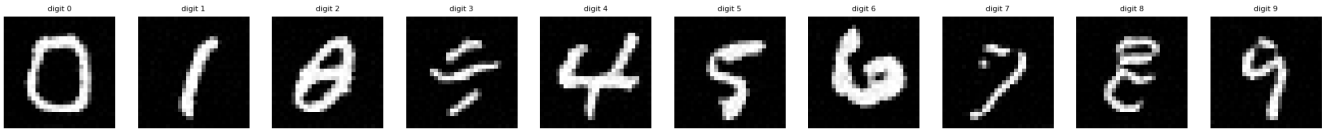

Figure 7: Photos sampled by DDPM with digit prompt

According to the plot above, the model performs worse with prompts like "digit four". The numbers 0, 1, 4, 5, 6, 7, 9 look particularly clear. At the same time, the digits 2, 3 and 6 have a slightly blurred or distorted shape.

Consider the results of the evaluation metrics:

- **FID Score:** 108.33
- **Inception Score:** 1.7410 ± 0.0667
- **PRDC:**
  - Precision: 0.2370
  - Recall: 0.1490
  - Density: 0.1550
  - Coverage: 0.2370

- **CLIP Score:** 24.42

The metrics indicate that **DDPM with digit prompts** performs significantly better than with long prompts. It achieves a relatively low **FID Score** of 108.33 and a solid **Inception Score** of 1.7410 ± 0.0667, suggesting decent visual quality of generated digits. The **PRDC metrics** (Precision: 0.2370, Recall: 0.1490, Density: 0.1550, Coverage: 0.2370) confirm improved accuracy and diversity of samples compared to long-prompt settings. Additionally, the **CLIP Score** reaches 24.42, which is the highest among all tested models, indicating the best alignment between text prompts and generated images.

5.1.3 DDPM with numeric prompt

Let's look at the results of the trained DDPM model with prompts: *"1"*.

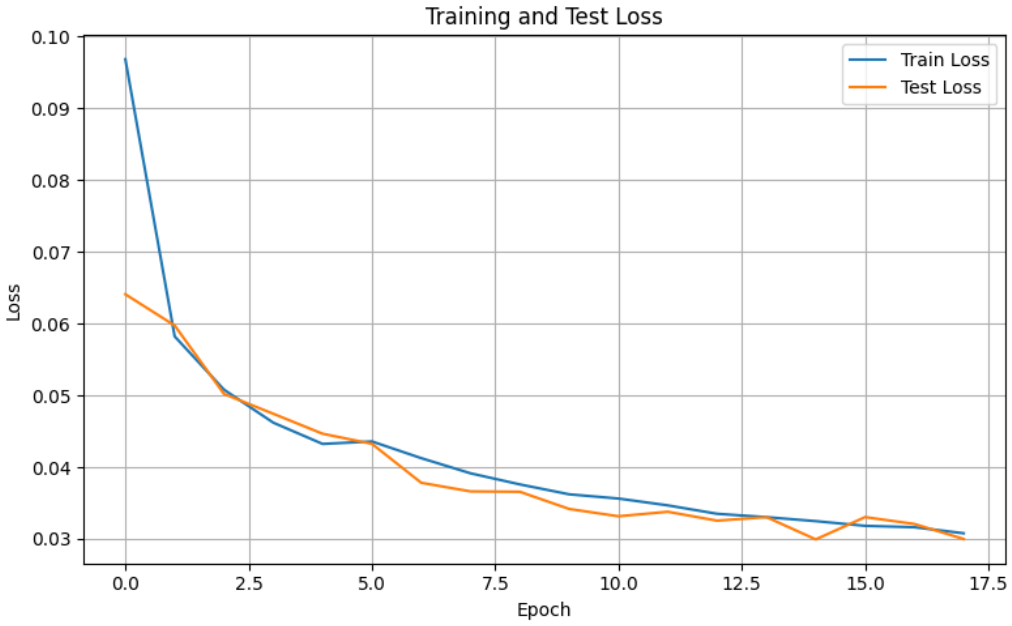

Figure 8: Loss during the training DDPM with numeric prompt

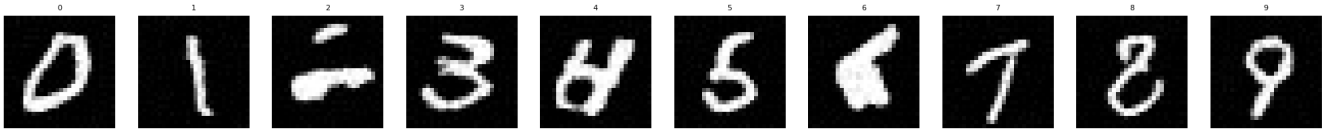

Figure 9: Photos sampled by DDPM with numeric prompt

Numeric prompts are the shortest and do not contain any context. As a result, the model receives minimal semantic information, which makes it difficult to accurately generate images, especially for visually similar or more complex numbers, like "6". However, there are photos with clear digits.

Let us now consider the results of the evaluation metrics:

- **FID Score:** 139.47
- **Inception Score:** 1.5623 ± 0.0651

- **PRDC:**
  - Precision: 0.1950
  - Recall: 0.0180
  - Density: 0.1274
  - Coverage: 0.1950
- **CLIP Score:** 23.43

The results of the **DDPM model with numeric prompts** show moderate performance. The model achieves a **FID Score** of 139.47 and an **Inception Score** of 1.5623 ± 0.0651, indicating an average visual similarity between real and generated images. **PRDC metrics** (Precision: 0.1950, Recall: 0.0180, Density: 0.1274, Coverage: 0.1950) suggest that while some digits are well-formed, the overall diversity and recall are limited. The **CLIP Score** of 23.43 confirms partial alignment with the textual input, though the lack of semantic richness in numeric prompts (`"0"`–`"9"`) likely restricted the model's capacity to generalize effectively.

5.1.4 DDIM with long prompt

Let's look at the results of the trained DDIM model with prompts: *"a handwritten digit two"*.

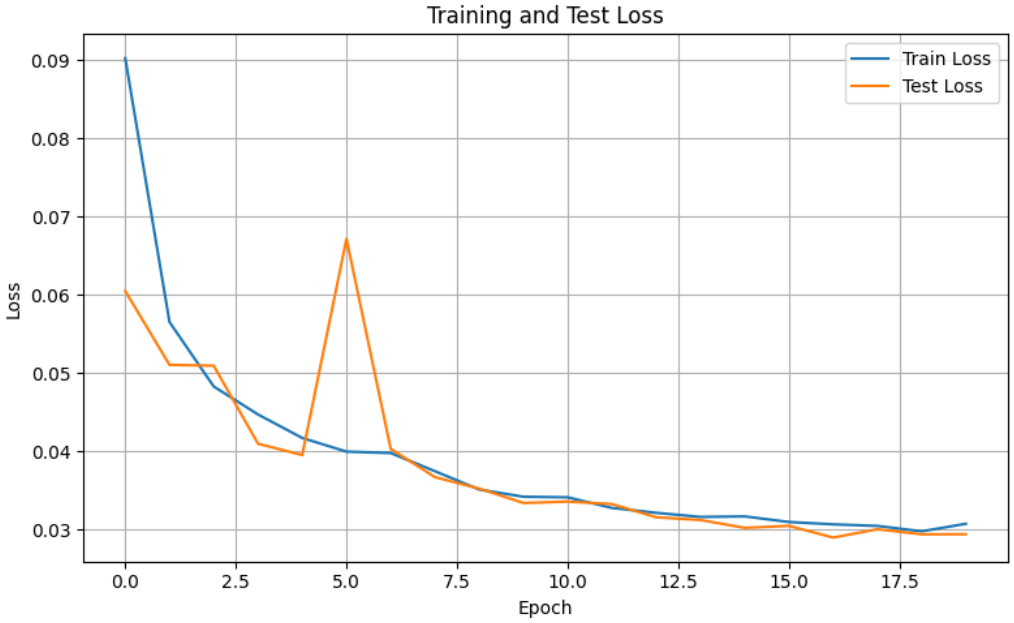

Figure 10: Loss during the training DDIM with long prompt

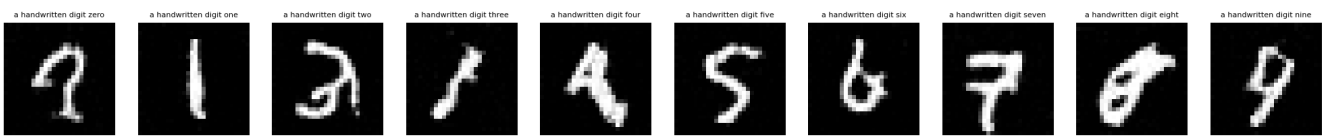

Figure 11: Photos sampled by DDIM with long prompt

Despite the fact that the prompt is the longest and carries the most information, the results are not the best. As in DDPMs, there are some digits, which are not easily recognizable to humans.

Let us now consider the results of the evaluation metrics:

- **FID Score:** 187.78
- **Inception Score:** 1.3699 ± 0.0247
- **PRDC:**
  - Precision: 0.0210
  - Recall: 0.0030
  - Density: 0.0156
  - Coverage: 0.0210
- **CLIP Score:** 25.36

The **DDIM model trained with long prompts** (*e.g., "a handwritten digit two"*) demonstrates the weakest results among all configurations. It obtained the **highest FID Score** (187.78), indicating a significant gap between the real and generated distributions. The **Inception Score** is also low (1.3699 ± 0.0247), reflecting poor sample quality and class distinguishability. The **PRDC metrics** (Precision: 0.0210, Recall: 0.0030, Density: 0.0156, Coverage: 0.0210) highlight very low diversity and coverage, and an inability to generate realistic images for all prompts.

Interestingly, although the **CLIP Score** of 25.36 is the highest among DDIM variants, indicating a relatively strong alignment with textual input, this did not translate into better visual quality. When compared to the **DDPM model with the same prompt type**, which achieved a lower FID (171.04) and slightly better PRDC scores, it becomes clear that the deterministic nature of DDIM did not cope well with long, detailed prompts that contain excess semantic load.

5.1.5 DDIM with digit prompt

Let's look at the results of the trained DDIM model with prompts: *"digit two"*.

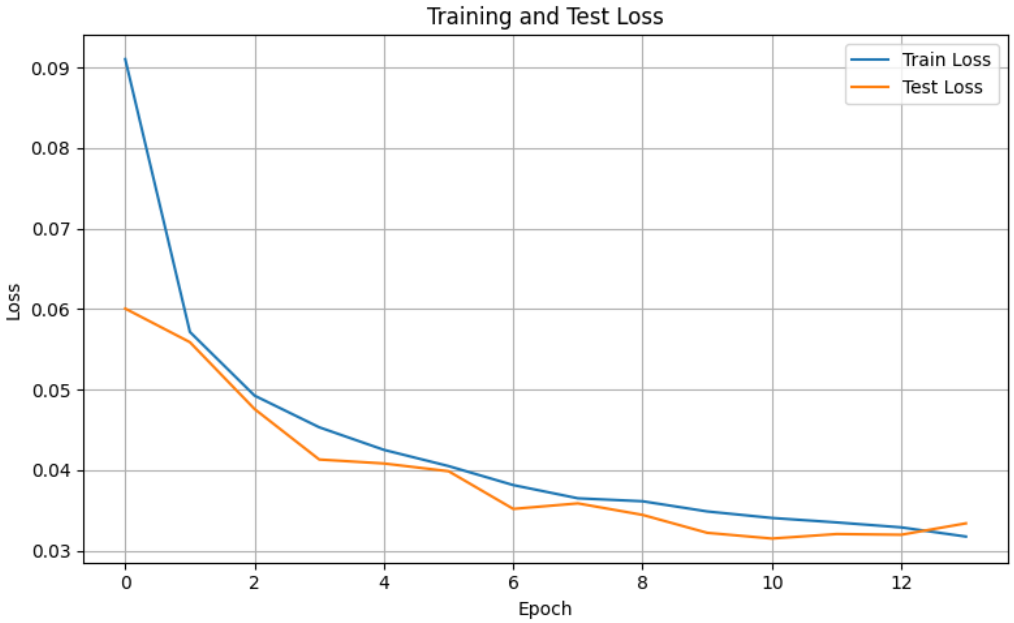

Figure 12: Loss during the training DDIM with digit prompt

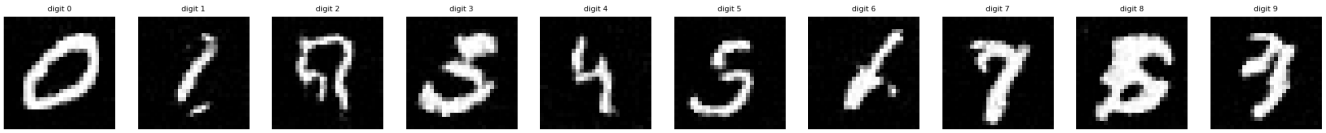

Figure 13: Photos sampled by DDIM with digit prompt

Digit-prompts are somewhat more informative than numeric ones (since they have the prefix "digit"), but still do not carry a rich context. This allows the model to generate more meaningful images than in the case of short numeric tokens, but some classes like 1, 2, 6, 8, 9 are not clear.

Let us now consider the results of the evaluation metrics:

- **FID Score:** 122.54
- **Inception Score:** 1.6236 ± 0.0630
- **PRDC:**
  - Precision: 0.1200
  - Recall: 0.1240
  - Density: 0.0728
  - Coverage: 0.1200
- **CLIP Score:** 23.91

The **DDIM model trained with digit prompts** (e.g., "digit two") shows a solid balance between semantic alignment and generation quality. It achieved a relatively low **FID Score** of 122.54 and an **Inception Score** of 1.6236 ± 0.0630, indicating that the model generates diverse and distinguishable samples. The **PRDC metrics** (Precision: 0.1200, Recall: 0.1240, Density: 0.0728, Coverage: 0.1200) show a decent level of realism and diversity, though not outstanding. The **CLIP Score** is 23.91, suggesting a moderate semantic correspondence with the input prompt.

Compared to the **DDPM model with digit prompts**, which achieved better metrics in almost all categories (FID: 108.33, IS: 1.7410 ± 0.0667, CLIP: 24.42, PRDC Precision: 0.2370), DDIM performs slightly worse. Thus, while the DDIM-digit variant is competent, the DDPM-digit model remains superior in both fidelity and semantic consistency for this prompt type.

5.1.6 DDIM with numeric prompt

Let's look at the results of the trained DDIM model with prompts: *"7"*.

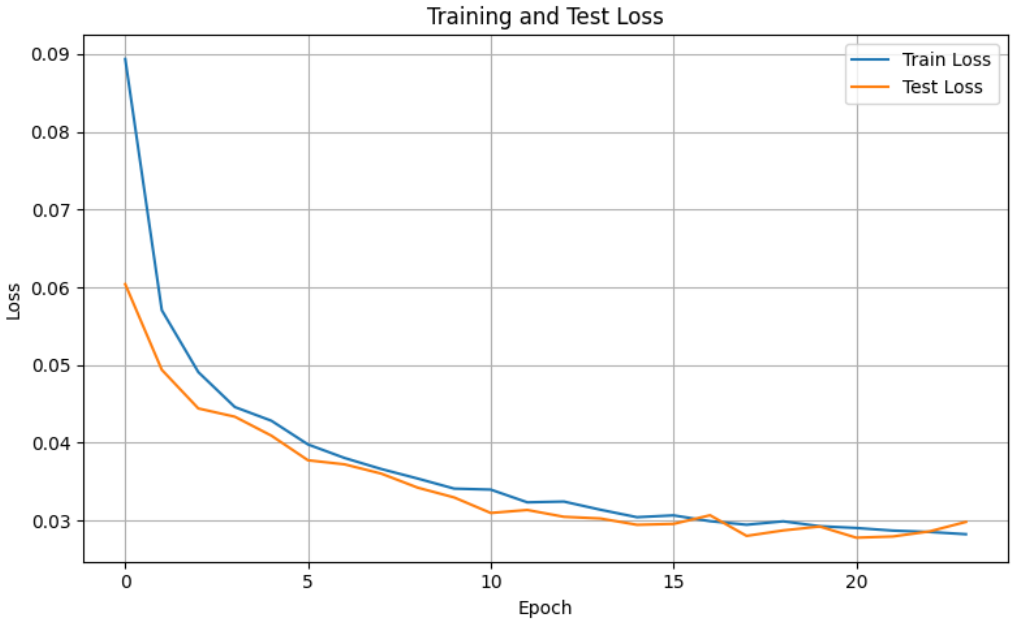

Figure 14: Loss during the training DDIM with numeric prompt

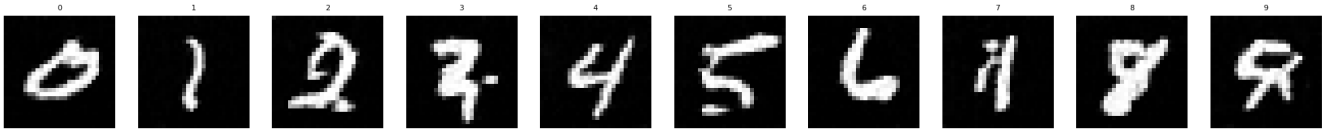

Figure 15: Photos sampled by DDIM with numeric prompt

As in the DDPMs, in DDIMs numeric prompts are the shortest, which makes it difficult to generate high-quality images. But in this case, even the digit "2" is recogniseable, while the model with other prompts could not cope with this image.

Let us now consider the results of the evaluation metrics:

- **FID Score:** 114.6852

- **Inception Score:** 1.4852 ± 0.0417
- **PRDC:**
  - Precision: 0.2520
  - Recall: 0.1590
  - Density: 0.1838
  - Coverage: 0.2520
- **CLIP Score:** 23.7

The **DDIM model trained with numeric prompts** (e.g., "7") demonstrates the best overall performance among all evaluated models. It achieves the **lowest FID Score** of 114.69 and a strong **Inception Score** of 1.4852 ± 0.0417, indicating a good balance of sample fidelity and diversity. Most notably, it achieves the **highest PRDC values** across all experiments (Precision: 0.2520, Recall: 0.1590, Density: 0.1838, Coverage: 0.2520), confirming that the generated digits are both accurate and varied. The **CLIP Score** of 23.7, while not the highest, still reflects decent alignment between image and prompt despite the lack of rich context.

Compared to the **DDPM model with numeric prompts** (FID: 139.47, IS: 1.5623 ± 0.0651, PRDC Precision: 0.1950), the DDIM variant clearly outperforms in all key quality metrics, particularly in PRDC. Thus, the DDIM-numeric combination proves to be the most robust setup for generating high-quality digit images.

5.1.7 Conclusion for DDPMs and DDIMs

The **DDIM model trained on numeric prompts** showed the best overall results. It achieved the lowest **FID Score** of **114.69**, indicating the smallest distance between the distributions of generated and real images. In addition, it reached the highest **PRDC** values: **Precision** – 0.2520, **Recall** – 0.1590, **Density** – 0.1838, **Coverage** – 0.2520. These results reflect the high quality, diversity, and realism of the generated images. The **CLIP Score** was 23.70, which is notably high considering the brevity of the prompts.

The **DDPM model trained on digit prompts** achieved the highest **CLIP Score** of **24.42**, indicating a stronger alignment between text and generated images. It also obtained competitive results in terms of FID (**108.33**) and Inception Score (**1.7410** $\pm$ **0.0667**). However, it was slightly inferior to the DDIM-numeric model in terms of PRDC: **Precision** – 0.2370, **Recall** – 0.1490, **Coverage** – 0.2370.

Models trained on **long prompts**, despite containing more semantic information (e.g., "a handwritten digit three"), demonstrated the weakest performance across most metrics. For instance, **DDPM-long** achieved a **FID** of **171.04** and very low **Precision** of **0.0330**, suggesting difficulties in generating stable images from lengthy or overly formal descriptions. A similar pattern was observed with **DDIM-long**, which reached a **FID** of **187.78** and a **Recall** of only **0.0030**.

Considering all the metrics, the best combination is the **DDIM model with numeric prompts**, which provided the most balanced image generation: high-quality, diverse, and close to real-world examples. Models with **digit prompts** demonstrated better semantic alignment between text and image but were slightly inferior in terms of quantitative accuracy. In contrast, **long prompts**, despite their informativeness, appeared to complicate the generation process and led to significantly worse performance across multiple evaluation metrics.

5.2 LDM

5.2.1 Architecture

- **Data and Prompt Setup**

  The model is trained on the MNIST dataset, which contains 60,000 grayscale images of handwritten digits (28×28 pixels) with class labels ranging from 0 to 9. For conditional generation, each digit is described using a natural language prompt of the form:

  `"a handwritten digit zero"`, `"a handwritten digit one"`, ..., `"a handwritten digit nine"`.

  These text prompts are embedded using a pretrained Transformer-based encoder (BERT), which converts each sentence into a 256-dimensional vector used for conditioning the denoising process.

- **Model Overview**

  The overall pipeline follows a compress-then-generate paradigm: the autoencoder learns a latent representation of MNIST digits, and the diffusion model is trained to generate samples in this latent space conditioned on text. The full sampling process involves decoding the denoised latent vector into the final image. A visual summary is provided in Figure 16.

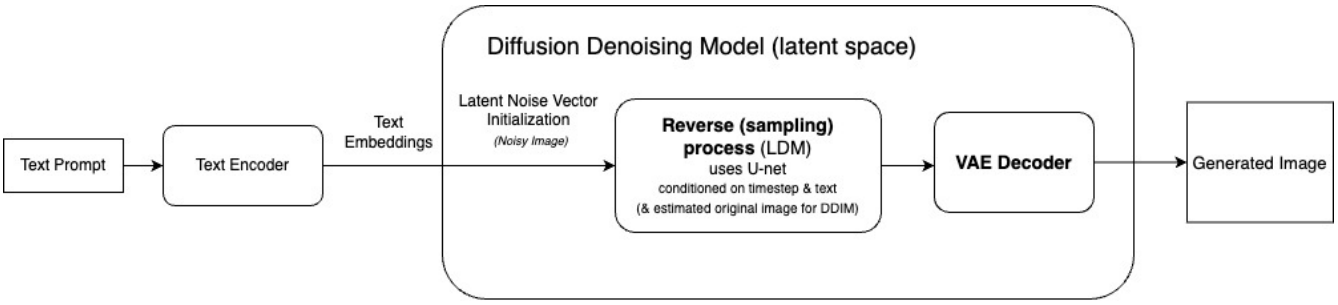

Figure 16: Architecture of the Latent Diffusion Model used in this work. The figure illustrates the generation pipeline: a text prompt is embedded via a text encoder, which conditions the denoising process in the latent space. A noisy latent vector undergoes iterative refinement through the reverse diffusion process (e.g., DDPM or DDIM), guided by the text embedding and timestep information. The final denoised latent is decoded by the VAE decoder into the output image.

- **Autoencoder Architecture**

  The autoencoder consists of a convolutional encoder $\mathcal{E}$ and a transposed-convolutional decoder $\mathcal{D}$. It maps each image $x \in R^{1\times 28\times 28}$ to a 16-dimensional latent vector $z \in R^{16}$. The encoder uses two convolutional layers followed by a fully connected layer, while the decoder mirrors this structure with one linear and two transposed convolutional layers.

  The autoencoder is trained using a reconstruction loss:

$$L_{\text{rec}} = E_{x\sim\mathcal{D}}\left[\|x - \mathcal{D}(\mathcal{E}(x))\|_2^2\right].$$

No adversarial training or latent space regularization (e.g., KL or VQ) is used, and the latent space is kept compact and continuous.

- **Latent Diffusion Process**

  After training the autoencoder, the encoder $\mathcal{E}$ is frozen and used to encode all training images into latent vectors $z_0 \in R^{16}$. A diffusion process is then defined in this latent space by progressively adding Gaussian noise over $T = 1000$ steps. The forward process is given by:

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

  where $\bar{\alpha}_t$ is the cumulative product of linearly scheduled noise coefficients.

  The reverse process is modeled using a denoising neural network $\varepsilon_\theta(z_t, t, e_y)$, where t is the (normalized) timestep and e_y is the embedded text prompt. The denoising objective is to predict the added noise:

$$L_{\text{LDM}} = E_{z_0,\, t,\, \varepsilon \sim \mathcal{N}(0,I)} \left[\|\varepsilon - \varepsilon_\theta(z_t, t, e_y)\|_2^2 \right].$$

- **Conditioning Mechanism**

  Conditioning is achieved by embedding the text prompt y into a fixed-length vector $e_y \in R^{256}$ using a pretrained text encoder. This embedding is linearly projected to match the internal dimension of the denoiser and is concatenated with time and noisy latent features before being passed through a multilayer perceptron.

- **Denoising Network Architecture**

  The denoiser ε_θ is a fully-connected network that takes as input the concatenated vector $[z_t \,\|\, \gamma(t) \,\|\, W_y e_y]$, where $\gamma(t)$ is a learned time embedding and $W_y \in R^{256 \times 64}$ is a projection matrix. The input is processed through two hidden layers with ReLU activations, and the final output is a 16-dimensional vector representing the predicted noise.

- **Sampling Procedure**

  To generate a digit conditioned on a prompt, the model samples $z_T \sim \mathcal{N}(0, I)$ and applies the reverse diffusion process from T to 0, conditioned on the prompt embedding. The final latent vector z_0 is decoded via $\mathcal{D}$ to produce the image:

$$\tilde{x} = \mathcal{D}(z_0).$$

- **Implementation Details**

  The model is implemented in PyTorch and trained on an MNIST-text dataset. The latent autoencoder is trained for 15 epochs using the Adam optimizer with learning rate 10^{-3}. The LDM is trained separately with early stopping (patience = 3) and the same optimizer settings.

5.2.2 LDM with long prompt

Reconstruction Quality of Autoencoder. Before training the diffusion model, a lightweight convolutional autoencoder is trained on the MNIST dataset to encode images into a compact 16-dimensional latent space. The quality of this encoding is critical, as

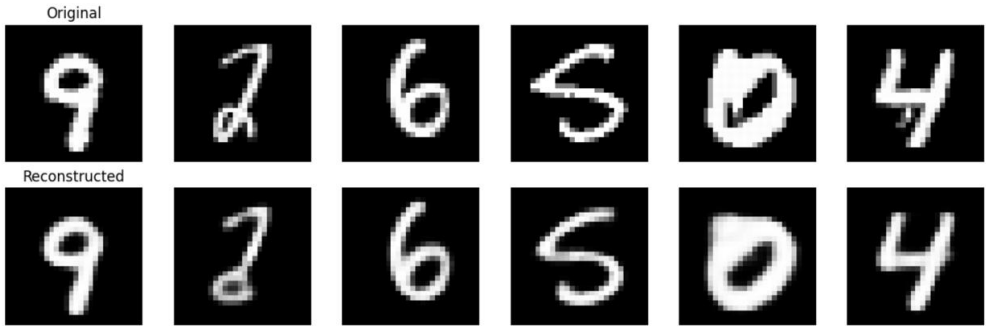

Figure 17: Autoencoder reconstruction performance: top row — original MNIST digits; bottom row — reconstructions. The model accurately preserves digit identity, enabling reliable training of the downstream LDM.

the diffusion process operates in this space. Figure 17 shows a side-by-side comparison of original digits and their reconstructions.

The reconstructions are clean and preserve the structural identity of the digits, confirming that the latent space learned by the autoencoder is sufficient for downstream generative modeling.

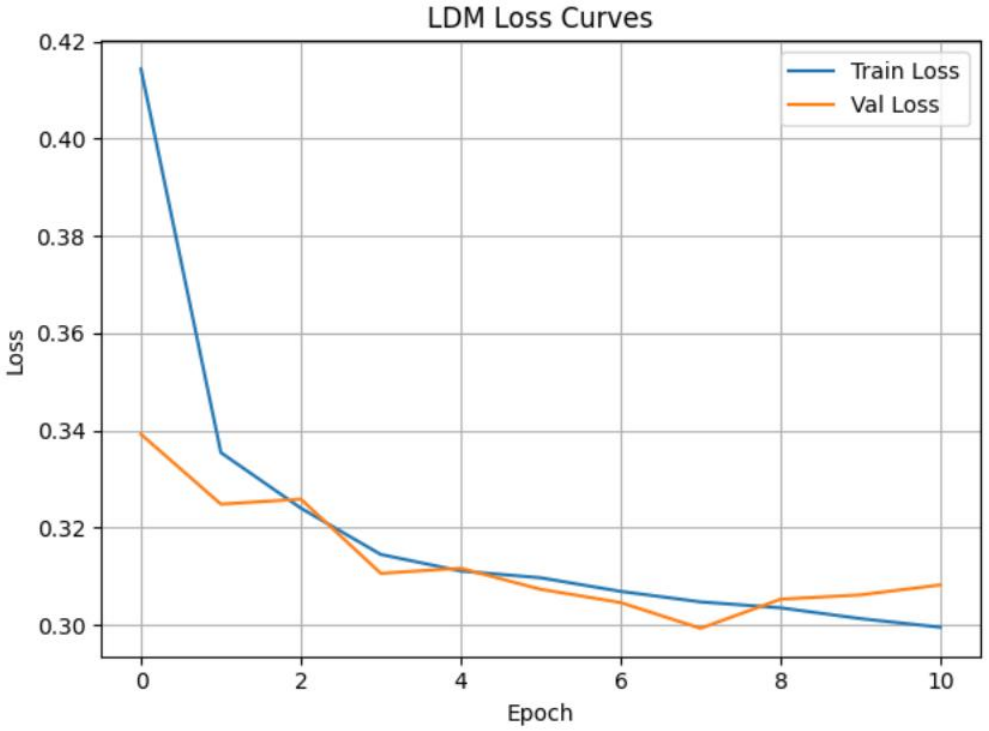

Figure 18: Loss during the training of LDM with long prompt

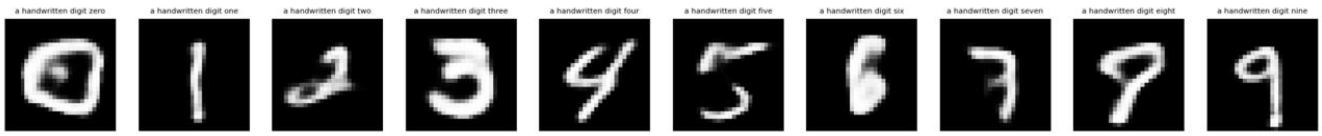

Figure 19: Images sampled by the Latent Diffusion Model (LDM) using long prompt

The generated digits generally capture the intended class, but there is still visible variability in quality across samples. Some outputs are sharp and realistic, while others are distorted. The reliance on latent space offers faster training and generation, though fine-grained details may be less well preserved compared to pixel-space diffusion models.

Evaluation Metrics. To assess the performance of the LDM quantitatively, we report the following scores:

- **FID Score:** 55.6817
- **Inception Score:** 2.1475 ± 0.1253
- **PRDC:**
  - Precision: 0.3380
  - Recall: 0.4290
  - Density: 0.2410
  - Coverage: 0.3380
- **CLIP Score:** 26.69

So, what can be concluded here is that the FID Score of 55.68 marks a substantial improvement over previous setups (e.g., DDIM or DDPM), suggesting that the generated images are statistically closer to the real MNIST distribution when measured via Inception feature space. This confirms the effectiveness of operating in a structured latent space for generating visually coherent samples.

Precision = 0.338 and Recall = 0.429 indicate that the generated samples not only align well with the real data distribution (high recall) but also maintain moderate sample quality (precision). Density (0.241) and Coverage (0.338) further support the observation that the LDM model successfully spans a meaningful subset of the real data manifold while preserving diversity. The CLIP Score of 26.69 (computed under long prompt conditioning) shows strong semantic alignment between generated images and their textual descriptions.

5.3 Comparison of all models by evaluation metrics

Table 1: Comparison of all models by evaluation metrics (best values in bold)

Model (Prompt)	FID	IS	Precision	Recall	CLIP Score
DDPM (long)	171.04	1.4758 ± 0.0410	0.0330	0.0570	23.99
DDPM (digit)	108.33	1.7410 ± 0.0667	0.2370	0.1490	24.42
DDPM (numeric)	139.47	1.5623 ± 0.0651	0.1950	0.0180	23.43
DDIM (long)	187.78	1.3699 ± 0.0247	0.0210	0.0030	25.36
DDIM (digit)	122.54	1.6236 ± 0.0630	0.1200	0.1240	23.91
DDIM (numeric)	114.69	1.4852 ± 0.0417	0.2520	0.1590	23.70
LDM (long)	**55.68**	**2.1475 ± 0.1253**	**0.3380**	**0.4290**	**26.69**

The **Latent Diffusion Model (LDM)** trained with long prompts significantly outperforms all other configurations across all evaluation criteria. It achieves a **FID score of 55.68**, which is substantially lower than any other model—indicating that its generated images are more statistically similar to the real MNIST digits. In contrast, the lowest FID among pixel-space diffusion models is 108.33 (achieved by DDPM with digit prompts), highlighting the strength of modeling in latent space.

In terms of **Inception Score (IS)**, which balances sample diversity and class clarity, LDM again leads with a score of **2.15 ± 0.13**. This marks a substantial improvement over the next best model, DDPM (digit), which achieves an IS of 1.74. The higher score reflects LDM's ability to generate digits that are both diverse and clearly classifiable.

The **PRDC metrics**, offering a more granular view of the generated distribution's quality and coverage, also favor the LDM. It reaches a **Precision of 0.338** and a **Recall of 0.429**, demonstrating both high sample quality and excellent coverage of the data manifold. The best-performing DDIM configuration (numeric prompt) trails behind with Precision and Recall scores of 0.252 and 0.159, respectively. Similarly, while DDPM (digit) performs relatively well with a Precision of 0.237, it cannot match LDM's comprehensive performance.

Finally, the **CLIP Score**, which captures semantic alignment between the generated images and their conditioning prompts, is again highest for LDM, at **26.69**. This suggests that not only are the LDM's outputs visually plausible, but they are also highly consistent with the descriptive language used during conditioning. Even DDIM (long), despite being explicitly conditioned on long prompts, achieves a lower CLIP score of 25.36.

6 Bibliography

References

[1] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, Bjorn Ommer *High-Resolution Image Synthesis with Latent Diffusion Models*. https://arxiv.org/pdf/2112.10752

[2] Patrick Esser, Robin Rombach, and Bjo rn Ommer. *Taming transformers for high-resolution image synthesis*. https://arxiv.org/pdf/2012.09841

[3] Jonathan Ho, Ajay Jain, Pieter Abbeel *Denoising Diffusion Probabilistic Models* https://arxiv.org/pdf/2006.11239v2

[4] Chenshuang Zhang, Chaoning Zhang, Mengchun Zhang, In So Kweon, Junmo Kim *Text-to-image Diffusion Models in Generative AI: A Survey* https://arxiv.org/pdf/2303.07909v3

[5] Jiaming Song, Chenlin Meng, Stefano Ermon *Denoising Diffusion Implicit Models*. https://arxiv.org/pdf/2010.02502

[6] Rui Cheng. PyTorch Text-to-Image GAN and Diffusion Models (Ollama).

https://github.com/RuiCheng19950208/RayCheng$_{P}yTorch_{T}ext2Img_{O}llama_{G}AN_{D}iffusion$

[7] Link to the repository (author's implementation of DDPM, https://arxiv.org/pdf/2112.10752